CS315 · ALGORITHMS ANALYSIS & DESIGN

Lecture 3 Dictionaries & Hash Tables

ملخص الميد · Goodrich 2.5

محارب

محتوى الملف

01 الفكرة

02 Dictionary ADT

03 Log File

04 Hash Function و Hash Table

05 Hash Code Maps

06 Compression Maps

07 Collisions و Chaining

08 Linear Probing

09 البحث والحذف مع Linear Probing

10 Double Hashing

11 الأداء و Load Factor و Rehashing

12 ورقة المراجعة

01 الفكرة

الـ **Dictionary** مجموعة عناصر، كل عنصر له **key** نبحث به. نبي نسوي بحث وإضافة وحذف بأسرع شي. الـ **Hash table** يحوّل الـ key إلى رقم خانة في مصفوفة، فنروح للخانة مباشرة بدل ما ندوّر.

وش يجي منها في الميد

سؤال ثابت في ميد 472 و 451: دخّل مفاتيح في hash table حجمه 9 بالـ **chaining**، وفي 451 زاد عليه **double hashing**. وفي 442 سأل: اذكر طرق حل الـ collision، ومتى نقول الـ hash function «good».

02 Dictionary ADT

The dictionary ADT models a searchable collection of key-element items. The main operations are searching, inserting, and deleting items. Multiple items with the same key are allowed.

Method	وش تسوي
`findElement(k)`	ترجع العنصر اللي مفتاحه `k`، وإذا مو موجود ترجع `NO_SUCH_KEY`.
`insertItem(k, o)`	تضيف العنصر `(k, o)`.
`removeElement(k)`	تحذف العنصر اللي مفتاحه `k` وترجعه، وإذا مو موجود ترجع `NO_SUCH_KEY`.
`size(), isEmpty(), keys()`	معلومات عن الـ dictionary.

تطبيقات: credit card authorization، address book، وتحويل host names إلى internet addresses.

03 Log File

أبسط تطبيق للـ dictionary: نخزن العناصر في **unsorted sequence** بأي ترتيب.

Method	الوقت	ليش
`insertItem`	`O(1)`	نضيف في البداية أو النهاية مباشرة.
`findElement`	`O(n)`	في أسوأ حالة (العنصر مو موجود) نمر على الكل.
`removeElement`	`O(n)`	لازم نلقاه أول.

يصلح بس للـ dictionaries الصغيرة، أو لما الإضافة كثيرة والبحث والحذف نادر، مثل سجل تسجيل الدخول.

04 Hash Function و Hash Table

A hash function h maps keys of a given type to integers in a fixed interval [0, N – 1].
Example: h(x) = x mod N is a hash function for integer keys.
The integer h(x) is called the hash value of key x.

الـ **Hash table** يتكون من شيئين: hash function `h`، ومصفوفة (table) حجمها `N`. الهدف نخزن العنصر `(k, o)` في الخانة `i = h(k)`.

مثال المحاضرة: نخزن `(SSN, Name)` في مصفوفة حجمها `N = 10,000` والدالة `h(x)` = آخر أربع خانات من `x`. فالرقم `451-229-0004` يروح الخانة 4.

الـ hash function تتكون من دالتين ورا بعض:

```
Hash code map:     h₁ : keys → integers
```

```
Compression map:   h₂ : integers → [0, N − 1]
```

```
h(x) = h₂(h₁(x))
```

متى تكون الـ hash function جيدة (good)؟

لما توزّع المفاتيح على الخانات بشكل عشوائي ومتساوي (**disperse**)، فتقل الـ collisions. وتكون سريعة في الحساب.

05 Hash Code Maps

الطريقة	الفكرة	تصلح لـ
Memory address	نعتبر عنوان الـ object في الذاكرة رقم.	جيدة عموماً، إلا للمفاتيح الرقمية والنصية.
Integer cast	نعتبر bits المفتاح رقم صحيح.	مفاتيح طولها ≤ طول الـ int (byte, short, int, float).
Component sum	نقسم الـ bits إلى أجزاء ثابتة الطول ونجمعها.	مفاتيح رقمية أطول من الـ int (long, double).
Polynomial accumulation	نحسب polynomial من الأجزاء عند قيمة ثابتة `z`.	النصوص (strings).

$$p(z) = a_0 + a_1 z + a_2 z^2 + \ldots + a_{n-1} z^{n-1}$$

- الـ **Component sum** مو جيد للنصوص، لأن الكلمات اللي لها نفس الحروف بترتيب مختلف (مثل tops و pots) تعطي نفس المجموع.
- اختيار `z = 33` يعطي 6 collisions على الأكثر في 50,000 كلمة إنجليزية.
- الـ polynomial ينحسب في `O(n)` بـ **Horner's rule**.

06 Compression Maps

الطريقة	المعادلة	الشرط
Division	$h_2(y) = y \bmod N$	`N` يُختار عدد أولي (prime) غالباً.
MAD Multiply, Add and Divide	$h_2(y) = (ay + b) \bmod N$	`a` و `b` أعداد غير سالبة، ولازم `a mod N ≠ 0`.

ليش الشرط؟ لو `a mod N = 0` كل المفاتيح تروح لنفس الخانة `b`. مثال المحاضرة:

```
N = 5,  a = 15,  b = 2
```

```
h₂(3) = (45 + 2) mod 5  = 47 mod 5  = 2
```

```
h₂(7) = (105 + 2) mod 5 = 107 mod 5 = 2
```

```
always 2, because a mod N = 0
```

07 Chaining و Collisions

Collisions occur when different elements are mapped to the same cell.

الطريقة	النوع	الفكرة
Chaining	خارج الجدول	كل خانة تأشر على linked list فيها كل العناصر اللي جتها.
Linear probing	**Open addressing**	نحط العنصر في أول خانة فاضية بعدها.
Double hashing	**Open addressing**	نقفز بخطوة تحددها دالة ثانية `d(k)` .

الـ **Chaining** بسيط، لكنه يحتاج ذاكرة إضافية خارج الجدول. في الـ **Open addressing** العنصر المتصادم ينحط في خانة ثانية داخل الجدول نفسه.

Ex مثال المحاضرة: Chaining

Draw the 7-item hash table resulting from hashing keys 12, 44, 13, 55 and 5, using the hash function h(i) = (3i + 2) mod 7, and assume that collisions are handled by chaining.

Key	12	44	13	55	5
(3i+2) mod 7	3	1	6	6	3

```
h(12) = 38 mod 7 = 3        h(44) = 134 mod 7 = 1
```

```
h(13) = 41 mod 7 = 6        h(55) = 167 mod 7 = 6
```

```
h(5)  = 17 mod 7 = 3
```

Index	Chain
0	∅
1	44
2	∅
3	12 → 5
4	∅
5	∅
6	13 → 55

المفتاح الجديد ينضاف في **آخر** السلسلة، مثل ما يطلب سؤال الميد.

Linear Probing 08

لو الخانة `h(k)` مشغولة نجرب اللي بعدها، وبعدها، ونلف من آخر الجدول لأوله (circularly). كل خانة نفحصها اسمها **probe**.

```
probe j:   (h(k) + j) mod N        for j = 0, 1, 2, …
```

Ex مثال المحاضرة

h(x) = x mod 13. Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order.

Key	k mod 13	Probes	Cell
18	5	5	**5**
41	2	2	**2**
22	9	9	**9**
44	5	5 → 6	**6**
59	7	7	**7**
32	6	6 → 7 → 8	**8**
31	5	5 → 6 → 7 → 8 → 9 → 10	**10**
73	8	8 → 9 → 10 → 11	**11**

0	1	2	3	4	5	6	7	8	9	10	11	12
		41			18	44	59	32	22	31	73	

عيبه: العناصر المتصادمة تتكتل جنب بعض (**lump together**)، فالتصادمات الجاية تحتاج probes أطول.

شف 31: احتاج 6 probes.

09 البحث والحذف مع Linear Probing

findElement(k): نبدأ من الخانة `h(k)` ونفحص خانة ورا خانة لين يصير واحد من ثلاثة: نلقى المفتاح، أو نلقى خانة فاضية، أو نفحص `N` خانة بدون فايدة.

```
Algorithm findElement(k)
    i ← h(k)
    p ← 0
    repeat
        c ← A[i]
        if c = ø
            return NO_SUCH_KEY
        else if c.key() = k
            return c.element()
        else
            i ← (i + 1) mod N
            p ← p + 1
    until p = N
    return NO_SUCH_KEY
```

المشكلة في الحذف: لو فضّينا خانة، البحث عن عنصر بعدها بيوقف عندها ويقول مو موجود. الحل: نحط مكان المحذوف علامة خاصة اسمها **AVAILABLE**.

Method	وش تسوي
`removeElement(k)`	نبحث عن `k`. لو لقيناه نحط مكانه `AVAILABLE` ونرجّع العنصر، وإلا نرجّع `NO_SUCH_KEY`.
`insertItem(k, o)`	لو الجدول مليان نرمي exception. نبدأ من `h(k)` ونمشي لين نلقى خانة فاضية **أو** فيها `AVAILABLE`، ونخزن فيها.

مثال: نحذف 44 من جدول المحاضرة. الخانة 6 تصير `AV` (يعني AVAILABLE). البحث عن 31 يبدأ من 5 ويعدّي الخانة 6 عادي لأنها مو فاضية، ويلقاه في 10.

0	1	2	3	4	5	6	7	8	9	10	11	12
		41			18	AV	59	32	22	31	73	

Double Hashing 10

بدل ما نمشي خانة خانة، نقفز بخطوة `d(k)` تحسبها دالة ثانية. كذا كل مفتاح له خطوة مختلفة، فيقل التكتل.

```
probe j:   (i + j·d(k)) mod N        j = 0, 1, … , N − 1
```

```
i = h(k)
```

```
d(k) = q − (k mod q)                 q < N,  q is a prime
```

- `d(k)` ما يصير تساوي صفر أبداً.
- `N` لازم يكون prime عشان نقدر نمر على كل الخانات.
- القيم الممكنة لـ `d(k)` هي `1, 2, … , q`.

Ex مثال المحاضرة

N = 13, h(k) = k mod 13, d(k) = 7 − (k mod 7). Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order.

Key	h(k)	d(k)	Probes	Cell
18	5	3	5	**5**
41	2	1	2	**2**
22	9	6	9	**9**
44	5	5	5 → 10	**10**
59	7	4	7	**7**
32	6	3	6	**6**
31	5	4	5 → 9 → 0	**0**
73	8	4	8	**8**

0	1	2	3	4	5	6	7	8	9	10	11	12
31		41			18	32	59	73	22	44		

```
44:  h = 5 taken,  d = 7 − 2 = 5   →  (5 + 5) mod 13 = 10
```

```
31:  h = 5 taken,  d = 7 − 3 = 4   →  9 taken  →  (5 + 8) mod 13 = 0
```

Ex مثال المحاضرة الثاني

A hash table of size 11 with double hashing: H(k) = k mod 11, H2(k) = 5 − (k mod 5). Insert 16, 239, 34, 12, 56.

Key	k mod 11	5 − (k mod 5)	Probes	Cell
16	5	4	5	**5**
239	8	1	8	**8**
34	1	1	1	**1**
12	1	3	1 → 4	**4**
56	1	4	1 → 5 → 9	**9**

0	1	2	3	4	5	6	7	8	9	10
	34			12	16			239	56	

انتبه لدالة الميد

ميد 451 أعطى دالة ثانية بشكل مختلف: $h_2(k) = 1 + (k \bmod 5)$. الطريقة نفسها بالضبط، بس عوّض بالدالة اللي يعطيك السؤال.

11 الأداء و Load Factor و Rehashing

الحالة	الوقت	متى
Worst case	O(n)	لما كل المفاتيح تتصادم.
Expected	O(1)	لكل عمليات الـ dictionary.

$$\text{Load factor:} \quad \alpha = \frac{N}{M}$$

$$N = \text{number of keys inserted}, \quad M = \text{size of the table}$$

$$\text{Expected probes for an insertion (open addressing)} = \frac{1}{1 - \alpha}$$

α	0.25	0.5	0.75	0.9
$\frac{1}{1-\alpha}$	1.33	2	4	10

- الـ hashing سريع جداً عملياً، بشرط الـ load factor ما يقرب من 100%.
- الـ **Rehashing**: لما الجدول يمتلي نسوي جدول جديد حجمه الضعف وننقل العناصر له.
- تطبيقات ثانية: small databases، compilers، browser caches.

انتبه للرموز: في سلايد الـ load factor الحرف `N` يعني عدد المفاتيح و `M` حجم الجدول. في باقي المحاضرة `N` هو حجم الجدول.

12 ورقة المراجعة

السؤال	الجواب
Log file	insertItem O(1) · findElement O(n) · removeElement O(n)
Hash table	Expected O(1) · Worst O(n)
طرق حل الـ collision؟	Chaining · Linear probing · Double hashing
Linear probing	$(h(k) + j) \bmod N$
Double hashing	$(h(k) + j \cdot d(k)) \bmod N$, $d(k) = q - (k \bmod q)$
MAD	$(ay + b) \bmod N$, $a \bmod N \neq 0$
علامة الحذف؟	AVAILABLE
Load factor	$\alpha = \frac{N}{M}$, probes $= \frac{1}{1-\alpha}$